

Universidad del Valle de Guatemala

CC3067 - Redes

Proyecto 1

Uso de un protocolo existente: Model Context Protocol

Gestión de reservas de salas de reuniones

Estudiante: Anggelie Velásquez

Carné: 221181

Curso: Redes

Año: 2026

Índice

1. Introducción	2
2. Arquitectura general	2
3. Servidores MCP utilizados	2
3.1. Filesystem MCP	2
3.2. Git MCP	3
4. Servidor MCP propio: reservas de salas	3
4.1. Herramientas y parámetros	3
4.2. Implementación local	3
4.3. Implementación remota	4
5. Despliegue en Render	4
6. Implementación manual de MCP y JSON-RPC	5
6.1. Extracto real del log de aplicación	5
7. Análisis de comunicación con Wireshark	6
7.1. DNS	6
7.2. Capa de enlace	6
7.3. Capa de red	7
7.4. Capa de transporte	7
7.5. Capa de aplicación y TLS	7
8. Pruebas y validación	8
9. Dificultades y lecciones aprendidas	8
9.1. Dificultades	8
9.2. Lecciones aprendidas	8
10. Conclusiones	9

1. Introducción

El proyecto implementa un chatbot de línea de comandos capaz de utilizar servidores Model Context Protocol (MCP). La solución integra un modelo de lenguaje mediante API, conserva el contexto de la sesión y registra las solicitudes y respuestas intercambiadas con los servidores MCP.

Se utilizaron dos servidores MCP existentes, Filesystem MCP y Git MCP, y se desarrolló un servidor MCP propio para la gestión de reservas de salas de reuniones. El servidor propio funciona de dos maneras: localmente mediante `stdio` y remotamente mediante HTTP/HTTPS. La versión remota fue desplegada en Render.

Una restricción importante del proyecto fue implementar manualmente el formato e intercambio de mensajes MCP/JSON-RPC, sin utilizar FastMCP ni SDK de MCP. El SDK de Anthropic se utiliza únicamente para consumir la API del modelo de lenguaje.

2. Arquitectura general

La arquitectura está compuesta por los siguientes elementos:

- **Chatbot Host:** recibe las instrucciones del usuario en lenguaje natural y mantiene el contexto de la conversación.
- **Claude:** modelo de lenguaje utilizado para interpretar las solicitudes y decidir cuándo utilizar una herramienta.
- **McpManager:** administra los distintos clientes MCP, inicializa los servidores, consulta sus herramientas y enruta las llamadas.
- **Servidor MCP de reservas:** servidor desarrollado específicamente para este proyecto.
- **Filesystem MCP:** servidor oficial utilizado para operaciones controladas sobre archivos.
- **Git MCP:** servidor oficial utilizado para consultar y ejecutar operaciones sobre repositorios Git.
- **Logger MCP:** registra las interacciones cliente-servidor en formato JSON Lines.

Las herramientas se presentan al modelo usando un nombre que identifica tanto el servidor como la operación. Algunos ejemplos son `reservations__list_rooms`, `filesystem__write_file` y `git__git_status`.

3. Servidores MCP utilizados

3.1. Filesystem MCP

Se integró el servidor oficial `@modelcontextprotocol/server-filesystem`. Su acceso se restringe a un directorio de trabajo definido por el proyecto, evitando que el chatbot tenga acceso libre a todo el sistema de archivos.

Entre las herramientas observadas durante la negociación se encuentran `read_file`, `write_file`, `create_directory`, `list_directory`, `move_file`, `search_files` y `get_file_info`.

3.2. Git MCP

Se utilizó el servidor oficial `mcp-server-git`, ejecutado mediante `uvx`. El servidor expone operaciones como `git_status`, `git_add`, `git_commit`, `git_log`, `git_diff` y operaciones de ramas.

Durante la implementación se verificó que el servidor no proporciona una herramienta `git_init`. Por esa razón, la creación inicial del repositorio se realiza externamente mediante el comando `git init`; las operaciones posteriores sí se realizan mediante Git MCP.

4. Servidor MCP propio: reservas de salas

El caso de uso seleccionado consiste en administrar reservas de salas de reuniones. El servidor valida la existencia de las salas, los intervalos de tiempo y la superposición de reservas.

4.1. Herramientas y parámetros

Herramienta	Descripción y parámetros principales
<code>list_rooms</code>	Lista las salas disponibles y sus características. No requiere parámetros.
<code>check_availability</code>	Comprueba disponibilidad. Parámetros: <code>room_id</code> , <code>date</code> , <code>start_time</code> , <code>end_time</code> .
<code>create_reservation</code>	Crea una reserva. Parámetros: <code>room_id</code> , <code>date</code> , <code>start_time</code> , <code>end_time</code> , <code>reserved_by</code> , <code>title</code> .
<code>cancel_reservation</code>	Cancela una reserva existente mediante <code>reservation_id</code> .
<code>list_reservations</code>	Lista reservas y permite filtros opcionales por fecha o sala.

Las salas iniciales utilizadas en las pruebas son Sala Atitlán, Sala Pacífico y Sala Maya.

4.2. Implementación local

La ejecución local utiliza un transporte `stdio`. El chatbot inicia el servidor como un subproceso y envía una línea JSON-RPC por cada solicitud. El cliente correlaciona solicitudes y respuestas mediante el campo `id`.

El flujo de sincronización comienza con `initialize`; después de recibir la respuesta del servidor, el cliente envía `notifications/initialized`. A partir de ese momento puede solicitar `tools/list` y realizar operaciones con `tools/call`.

4.3. Implementación remota

La misma lógica del servidor fue expuesta mediante un servidor HTTP implementado con la biblioteca estándar de Python. Se reutiliza el mismo manejador de MCP/JSON-RPC, de forma que la lógica del protocolo no depende del transporte.

Los endpoints utilizados son:

- GET /health: comprobación básica del estado del servicio.
- POST /mcp: intercambio de mensajes MCP/JSON-RPC.

El endpoint público del servidor desplegado es:

<https://mcp-reservas.onrender.com/mcp>

El chatbot puede seleccionar la implementación local o remota mediante `MCP_RESERVATIONS_MODE`. Cuando se usa el modo remoto, `MCP_RESERVATIONS_URL` permite configurar la URL del servidor sin dejarla fija dentro de la lógica principal.

5. Despliegue en Render

El servidor remoto fue desplegado como Web Service en Render. Para que el servicio fuera accesible desde Internet se configuró el proceso para escuchar en `0.0.0.0` y utilizar el puerto suministrado por la variable `PORT`.

La Figura 1 muestra el despliegue exitoso y el mensaje de arranque del servidor en `0.0.0.0:10000`.

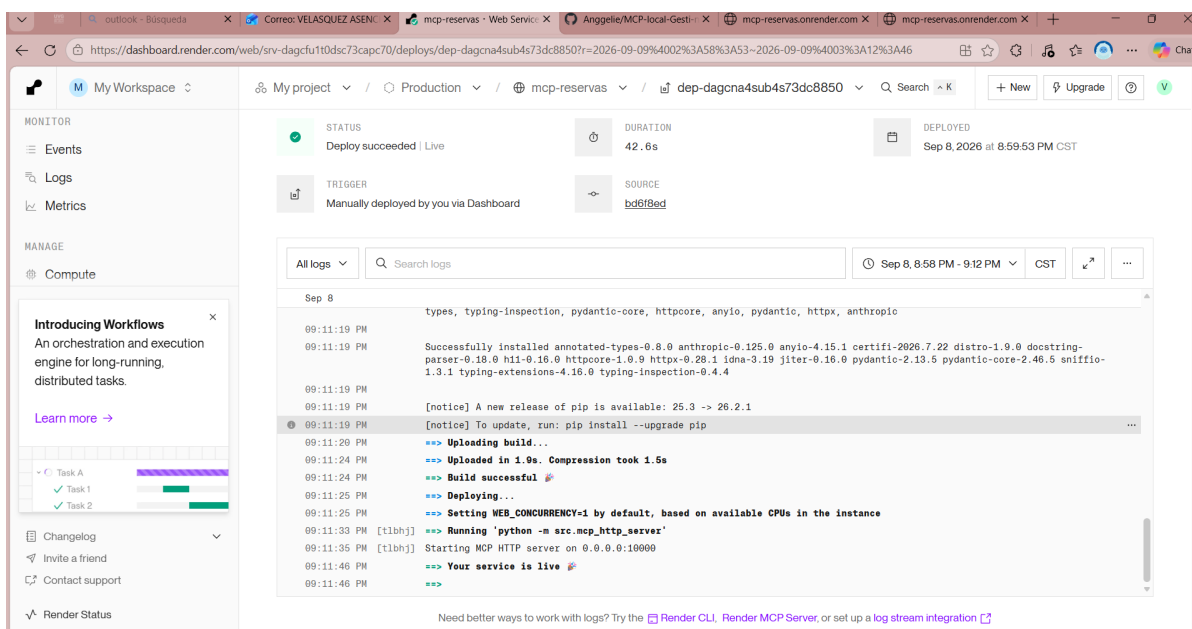


Figura 1: Despliegue exitoso del servidor MCP de reservas en Render.

El endpoint de salud respondió correctamente con `{"status": "ok"}`, como se observa en la Figura 2.

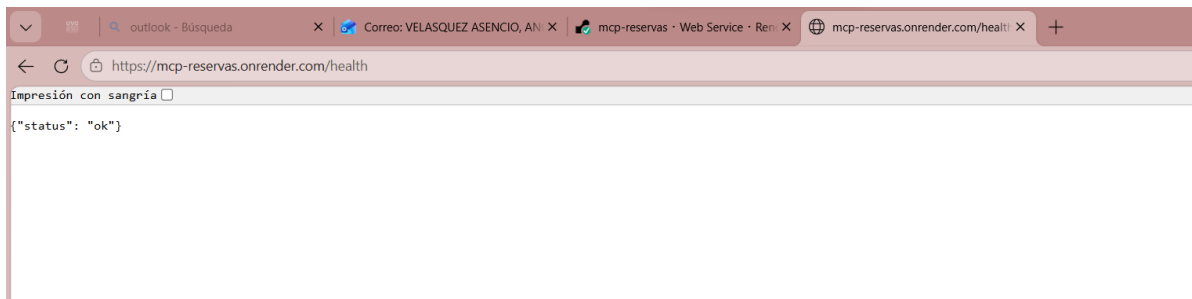


Figura 2: Comprobación del endpoint `/health` del servicio remoto.

6. Implementación manual de MCP y JSON-RPC

El cliente y el servidor propio construyen y validan manualmente los objetos JSON-RPC 2.0. No se utiliza una biblioteca que implemente MCP. Esto permite observar explícitamente los campos `jsonrpc`, `id`, `method`, `params`, `result` y los errores de protocolo.

La versión negociada durante las pruebas fue 2024-11-05. El flujo observado con el servidor remoto fue el siguiente:

Método	Clasificación	ID	Función
initialize	Sincronización / solicitud	1	Negocia versión, capacidades e información del cliente y servidor.
notifications/initialized	Sincronización / notificación	–	Confirma que la inicialización terminó; no espera respuesta.
tools/list	Solicitud	2	Solicita el catálogo de herramientas disponibles.
tools/call (list_rooms)	Solicitud	3	Ejecuta la herramienta para listar las salas.
tools/call (check_availability)	Solicitud	4	Consulta la disponibilidad de una sala.

Las respuestas JSON-RPC utilizan el mismo `id` de la solicitud correspondiente, permitiendo correlacionar cada petición con su respuesta. La notificación `notifications/initialized` no tiene `id`, ya que no requiere respuesta JSON-RPC.

6.1. Extracto real del log de aplicación

El siguiente extracto resume la secuencia registrada durante la demostración remota:

```
CLIENT_TO_SERVER id=1 method=initialize
SERVER_TO_CLIENT id=1 result={protocolVersion, capabilities, serverInfo}
CLIENT_TO_SERVER method=notifications/initialized
```

```

CLIENT_TO_SERVER id=2 method=tools/list
SERVER_TO_CLIENT id=2 result={tools: [...]}
CLIENT_TO_SERVER id=3 method=tools/call name=list_rooms
SERVER_TO_CLIENT id=3 result={content: [...]}
CLIENT_TO_SERVER id=4 method=tools/call name=check_availability
SERVER_TO_CLIENT id=4 result={available: true}

```

7. Análisis de comunicación con Wireshark

Para generar tráfico controlado se utilizó el script `scripts/remote_mcp_capture_demo.py`. Este script realiza únicamente operaciones de lectura contra el servidor remoto y registra de forma separada la secuencia JSON-RPC.

7.1. DNS

Antes de establecer la conexión HTTPS, el equipo resolvió el nombre `mcp-reservas.onrender.com`. La captura muestra una cadena CNAME hacia infraestructura de Render/Cloudflare y direcciones IPv4 216.24.57.7 y 216.24.57.15.

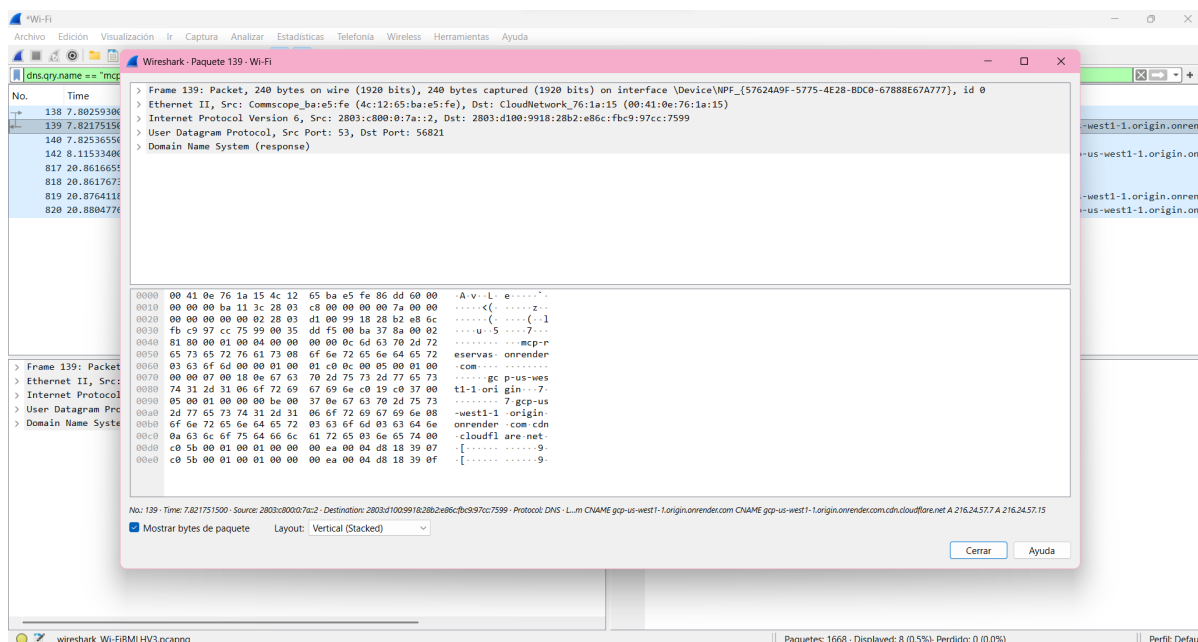


Figura 3: Respuesta DNS para `mcp-reservas.onrender.com`.

7.2. Capa de enlace

Wireshark presenta las tramas capturadas como Ethernet II. En esta capa aparecen direcciones MAC utilizadas para la entrega dentro del enlace local. Estas direcciones son relevantes únicamente para el salto local y no identifican directamente al servidor remoto de Render.

7.3. Capa de red

En la conexión aislada se observa IPv4 con origen 192.168.0.18, correspondiente al equipo cliente dentro de la red privada, y destino 216.24.57.7, obtenido mediante la resolución DNS del servicio remoto.

7.4. Capa de transporte

La comunicación utiliza TCP. En la captura seleccionada, el cliente utiliza el puerto efímero 50622 y el servidor utiliza el puerto 443. Se observa el establecimiento de conexión mediante el three-way handshake:

1. Cliente → servidor: SYN.
2. Servidor → cliente: SYN, ACK.
3. Cliente → servidor: ACK.

Posteriormente se observan los datos de la sesión y finalmente el cierre mediante FIN, ACK.

7.5. Capa de aplicación y TLS

El servicio remoto utiliza HTTPS. En la captura específica del MCP remoto se observa TLS 1.3, incluyendo Client Hello, Server Hello y múltiples registros Application Data. El SNI del Client Hello identifica mcp-reservas.onrender.com.

The screenshot shows a Wireshark capture of network traffic. The filter bar at the top is set to '(ip.addr == 216.24.57.7 || ip.addr == 192.168.0.18) && tcp.port == 443'. The packet list on the left shows several packets, with the selected packet being a TLSv1.3 Client Hello (packet 821). The packet details pane on the right shows the structure of the Client Hello, including the Change Cipher Spec, Application Data, and the TLSv1.3 handshake messages. The packet bytes pane at the bottom shows the raw data of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
821	20.883775380	192.168.0.18	216.24.57.7	TCP	66	50622 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
822	20.927767000	216.24.57.7	192.168.0.18	TCP	66	443 → 50622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM WS=8192
823	20.927929800	192.168.0.18	216.24.57.7	TCP	54	50622 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
824	20.929050100	192.168.0.18	216.24.57.7	TCP	1514	50622 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 825]
825	20.929050100	192.168.0.18	216.24.57.7	TLSv1.3	145	Client Hello (SNI=mcp-reservas.onrender.com)
826	20.972351100	216.24.57.7	192.168.0.18	TCP	60	443 → 50622 [ACK] Seq=1 Ack=1552 Win=131072 Len=0
827	20.972355800	216.24.57.7	192.168.0.18	TLSv1.3	2974	Server Hello, Change Cipher Spec
828	20.972356800	216.24.57.7	192.168.0.18	TLSv1.3	1026	Application Data
829	20.972492700	192.168.0.18	216.24.57.7	TCP	54	50622 → 443 [ACK] Seq=1552 Ack=3893 Win=65280 Len=0
830	20.976855100	192.168.0.18	216.24.57.7	TLSv1.3	134	Change Cipher Spec, Application Data
831	20.977072200	192.168.0.18	216.24.57.7	TLSv1.3	288	Application Data
832	20.977157500	192.168.0.18	216.24.57.7	TLSv1.3	272	Application Data
833	21.018525900	216.24.57.7	192.168.0.18	TCP	60	443 → 50622 [ACK] Seq=3893 Ack=2084 Win=131072 Len=0
841	21.310168100	216.24.57.7	192.168.0.18	TLSv1.3	1085	Application Data, Application Data
842	21.310169800	216.24.57.7	192.168.0.18	TLSv1.3	81	Application Data
843	21.310170400	216.24.57.7	192.168.0.18	TLSv1.3	78	Application Data
844	21.310171500	216.24.57.7	192.168.0.18	TCP	60	443 → 50622 [FIN, ACK] Seq=4975 Ack=2084 Win=131072 Len=0
845	21.310305600	192.168.0.18	216.24.57.7	TCP	54	50622 → 443 [ACK] Seq=2084 Ack=4976 Win=64256 Len=0

Figura 4: Flujo TCP y TLS 1.3 entre el cliente y el servidor MCP remoto.

El contenido JSON-RPC no aparece en texto plano en esta captura porque viaja protegido por TLS. Para identificar las interacciones MCP se correlacionó el tráfico de red con el log generado por el cliente. De esta manera se pudo clasificar `initialize` y `notifications/initialized` como mensajes de sincronización, `tools/list` y `tools/call` como solicitudes y los objetos con `result` como respuestas.

8. Pruebas y validación

La solución fue validada con pruebas automatizadas y demostraciones reales. Al cierre de la implementación se ejecutaron 51 pruebas con `unittest`, además de una compilación de los módulos de Python y una verificación de espacios y diferencias de Git.

Entre las validaciones realizadas se encuentran:

- negociación MCP con los tres servidores;
- descubrimiento y enrutamiento de herramientas;
- operaciones del servidor de reservas;
- integración real de Filesystem MCP;
- integración real de Git MCP;
- selección entre reservas local y remota;
- transporte HTTP hacia el servidor desplegado;
- manejo de errores controlados;
- generación del log de interacciones MCP.

9. Dificultades y lecciones aprendidas

9.1. Dificultades

Una de las principales dificultades fue implementar MCP/JSON-RPC manualmente y separar correctamente la lógica del protocolo de los distintos transportes. También fue necesario coordinar varios servidores MCP dentro del mismo chatbot sin duplicar su ciclo de inicialización.

En Windows se presentaron dificultades con la ejecución de `npx` y `uvx`, además del cierre de procesos hijos. Durante la integración de Git MCP se identificó que no existe una operación `git_init`, por lo que la inicialización del repositorio tuvo que mantenerse como una operación externa.

En el despliegue remoto fue necesario corregir la dirección de escucha para utilizar `0.0.0.0` y el puerto proporcionado por Render. En el análisis de Wireshark, el cifrado TLS impidió observar directamente el JSON-RPC en texto plano; por ello se utilizó el log de aplicación para correlacionar las operaciones MCP con el flujo HTTPS capturado.

9.2. Lecciones aprendidas

El proyecto permitió comprender el proceso de negociación de MCP, la correlación de mensajes JSON-RPC y la diferencia entre solicitudes, respuestas y notificaciones. También mostró que una misma lógica de protocolo puede utilizar transportes diferentes si la arquitectura separa correctamente responsabilidades.

A nivel de redes, la captura permitió relacionar una interacción de aplicación con DNS, IPv4, TCP y TLS. Esto hizo más clara la forma en que una operación de alto nivel, como consultar la

disponibilidad de una sala, depende de varias capas de comunicación antes de llegar al servidor remoto.

10. Conclusiones

Se implementó un chatbot funcional que integra tres servidores MCP: Filesystem, Git y un servidor propio para la gestión de reservas. El servidor desarrollado funciona tanto localmente como de forma remota y mantiene el mismo conjunto de herramientas en ambos modos.

La implementación manual de JSON-RPC permitió comprender de forma directa cómo MCP negocia capacidades, publica herramientas y ejecuta llamadas. La integración con Wireshark complementó esta implementación al mostrar el comportamiento real de la comunicación en las capas de enlace, red, transporte y aplicación.

Finalmente, el despliegue remoto en Render demostró que el mismo servidor MCP puede utilizarse a través de Internet mediante HTTPS sin cambiar la lógica funcional del chatbot, manteniendo la configuración del transporte separada de la lógica de negocio.